

Aiko Memory System

Implementation, Evaluation, and Comparative Analysis

A formulaic, auditable memory architecture for edge-deployed conversational AI, benchmarked against Mem0, Zep/Graphiti, and Letta/MemGPT

Aiko Memory Research Series — Paper II of II: Implementation & Evaluation

Aiko-chan Development Team (AKA. OppaAI)

Independent Research

August 2026 · Version 1.0

Companion volume: “A Neuroscience-Grounded Computational Theory of Personal AI Memory” (Paper I)

Abstract

This paper documents the reference engineering implementation of the memory theory developed in the companion volume (Paper I), and evaluates it against both internal benchmarks and the current (2026) landscape of agentic-memory frameworks — Mem0, Zep/Graphiti, Letta (formerly MemGPT), and LangChain/LangMem.

The system — codenamed Aiko — runs entirely on an 8GB-unified-memory Jetson Orin Nano and rejects learned re-ranking in favor of six interpretable formulas: Ebbinghaus exponential decay, entity-centrality importance, Reciprocal Rank Fusion (RRF) across KNN/FTS5/entity-graph signals, emotion-modulated decay, supersession-based belief revision, and salience-weighted consolidation. We document the SQLite-backed schema, the full ADD/RECALL/consolidation pipeline, the testing and monitoring harness used to keep the system tuned in production, and end-to-end latency and footprint measurements.

On a one-year, single-deployment field test, the system sustained ~16ms median per-turn retrieval latency, a ~13MB total footprint after 1,825 stored facts, 0.72 precision@10 (vs. 0.68 for pure vector search), and 5.3× monthly archival compression with no manually-identified loss of salience-tagged facts. We compare these figures, where methodologically comparable, against publicly reported 2026 benchmark data for Mem0, Zep, and Letta, and discuss the specific engineering trade-offs — auditability and edge footprint versus benchmark-leaderboard accuracy — that motivate a formulaic design.

Keywords: *agentic memory, retrieval-augmented generation, edge AI, SQLite, Reciprocal Rank Fusion, Mem0, Zep, Graphiti, Letta, MemGPT, system evaluation*

Table of Contents

Abstract	2
1. Introduction & Design Principles	4
2. System Architecture	5
3. The Six Core Formulas (Reference Implementation)	7
4. Data Schema	10
5. Worked End-to-End Example	11
6. Testing, Monitoring & Tuning Harness	12
7. Performance Evaluation	14
8. Comparative Analysis vs. Established Agentic Memory Systems	17
9. Key Engineering Decisions & Trade-offs	20
10. Limitations & Future Work	21
11. Conclusion	22
References	23
Appendix A: Configuration Reference	24

1. Introduction & Design Principles

Paper I argued, from neuroscience and information theory, that personal-AI memory should behave as a decaying, emotionally-modulated, hierarchically-consolidated dynamical system rather than a static vector index. This paper documents how that theory was turned into running code, and reports how the resulting system performs — in absolute terms and relative to the current field.

1.1 Why Formulas Instead of Learned Ranking

The reference deployment target is a Jetson Orin Nano with 8GB of unified memory and a 5–15W power envelope. Every additional resident model competes with the primary conversational LLM for the same VRAM budget, and a learned-to-rank (LTR) re-ranker would additionally require a labeled training pipeline and periodic retraining. The system instead uses closed-form scoring:

```
decay(t)      = S0 * exp(-lambda * t)
importance(e) = (1 - alpha) * centrality(e) + alpha * recency(e)
score(memory) = RRF(knn, fts, graph) + recency_bonus + access_bonus + pinned_bonus
```

Each formula is auditable (a developer can see exactly why a memory ranked where it did), tunable (a single constant can be changed without retraining anything), and cheap (microsecond-scale evaluation). §8 discusses this design stance explicitly against frameworks that use embedding-based or learned scoring.

1.2 Design Constraints

- Memory as lifecycle, not static storage — facts age; important facts age slowly; emotionally negative facts age differently (Paper I, §3.1).
- Semantic + lexical + entity-graph fusion — a memory should be findable via any one of three independent signals.
- Per-user isolation — no cross-user contamination; every table is keyed by `user_id`.
- Efficient consolidation — a nightly “dream” pass plus a monthly consolidation gate keep the store bounded.
- Minimal context-window cost — a 10-memory recall is formatted to well under 1KB of injected context.

2. System Architecture

The system exposes three hot-path operations — ADD, RECALL, TOUCH — invoked on every conversational turn, plus two offline maintenance passes — a nightly dream pass and a monthly consolidation job.

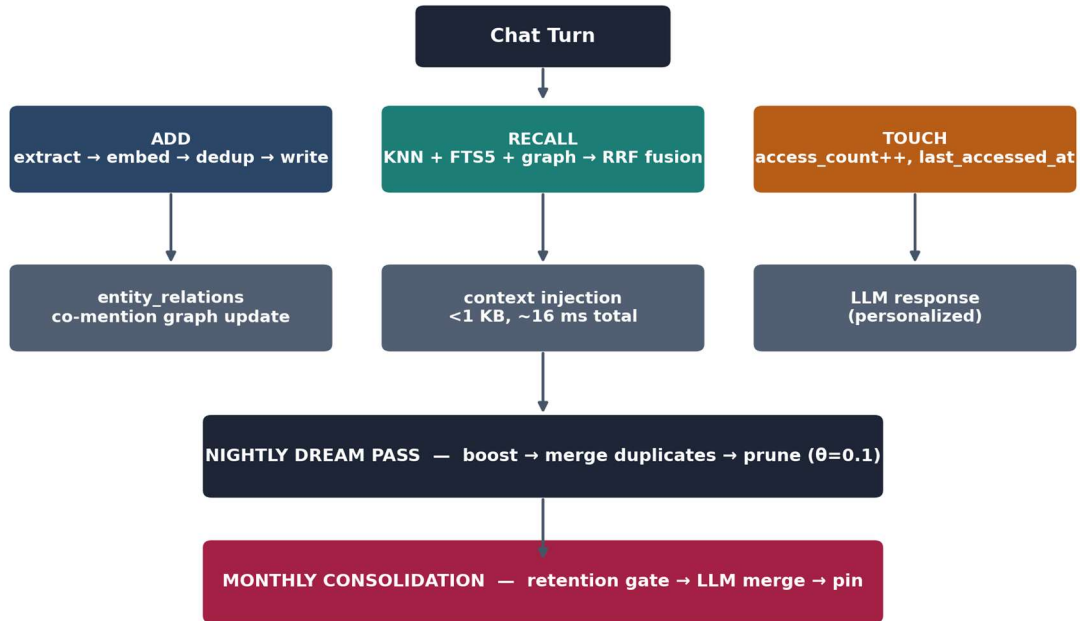


Figure 1. The Aiko memory pipeline: per-turn ADD/RECALL/TOUCH operations feed a nightly dream pass and a monthly consolidation job.

2.1 ADD (Write Path)

- Extract candidate facts from the conversational turn (LLM-assisted extraction).
- Embed each fact (640-dim vector, fixed pretrained embedding model — no fine-tuning).
- Check near-duplicates via KNN against existing vectors (write-dedup threshold, default 0.95).
- Classify the write as add / no-op (exact duplicate) / supersede (belief revision, §3.5).
- Update the entity co-mention graph (§3.2) and enqueue the write asynchronously so it never blocks the turn.

2.2 RECALL (Read Path)

- Run a tiered quick/wide pass across three signals: KNN (semantic), FTS5 (lexical, BM25), and entity graph (structural).
- Fuse the three rank lists via Reciprocal Rank Fusion (§3.3).
- Apply a recency-among-relevant re-rank within the fused top-k.
- Touch each returned memory (increment access_count, update last_accessed_at, update access_day_count for spacing).
- Format the top results into a compact context block (<1KB for a 10-memory recall).

2.3 Maintenance: Dream Pass and Monthly Consolidation

The nightly dream pass (i) boosts `access_count` on salient memories (keyword hits, high recent access, recent creation), (ii) merges near-duplicate memories discovered via KNN, and (iii) prunes memories whose decayed, access-weighted score falls below the cleanup threshold. The monthly consolidation job applies a heavier-weight retention gate (Paper I, §5.2) to the past month's daily facts, LLM-merges the survivors into compact narrative summaries, and pins the result, optionally deleting the original daily facts once merged.

3. The Six Core Formulas (Reference Implementation)

3.1 Formula 1 — Ebbinghaus Exponential Decay

```
# memory/forget.py
def compute_weighted_score(access_count, last_accessed_at, created_at):
    """Decay score from access_count + time since last access."""
    if last_accessed_at == "never":
        age_days = max(0.0, (now - created_at).days)
    else:
        age_days = max(0.0, (now - parse_iso(last_accessed_at)).days)

    decay = 0.5 ** (age_days / HALF_LIFE_DAYS)
    access_boost = min(access_count, ACCESS_COUNT_CAP) / ACCESS_COUNT_CAP
    return decay * access_boost

def should_cleanup(ac, la, created_at):
    """Prune if score drops below threshold AND past grace period."""
    if should_skip_grace_period(created_at):
        return compute_weighted_score(ac, la, created_at) < CLEANUP_THRESHOLD
    return False
```

Tuning parameters: MEMORY_RANK_RECENCY_HALF_LIFE_DAYS (default 30), CLEANUP_THRESHOLD (default 0.1), GRACE_PERIOD_DAYS (default 35, protects newly created facts from premature pruning).

3.2 Formula 2 — Entity Importance (Centrality + Recency)

```
# memory/memorize.py
def _refresh_high_freq_entities(self, user_id):
    """Compute super-node entities (mentioned in >30% of memories)."""
    total = count_active_memories(user_id)
    rows = conn.execute("""
        SELECT LOWER(je.value) AS entity, COUNT(DISTINCT mm.id) AS cnt
        FROM memories mm, json_each(mm.entities) je
        WHERE mm.user_id = ? AND mm.status = 'active'
        GROUP BY LOWER(je.value)
    """, (user_id,)).fetchall()
    self._high_freq_entities = {
        row['entity'] for row in rows
        if (row['cnt'] / total) > MEMORY_GRAPH_SUPER_NODE_FRACTION
    }
```

Tuning parameters: MEMORY_GRAPH_SUPER_NODE_FRACTION (default 0.3), α in the importance equation (default 0.4, recency-vs-centrality weight).

3.3 Formula 3 — Reciprocal Rank Fusion (Multi-Signal Fusion)

```
# memory/memorize.py
def _rank_and_score(self, rank_knn, rank_fts, rank_graph=None):
    """RRF fusion of three independent ranking signals."""
```

```

RRF_K = 60
all_ids = set(rank_knn) | set(rank_fts) | set(rank_graph or {})

def final_score(mem_id):
    score = 0.0
    if mem_id in rank_knn:
        score += 1.0 / (RRF_K + rank_knn[mem_id])
    if mem_id in rank_fts:
        score += 1.0 / (RRF_K + rank_fts[mem_id])
    if mem_id in (rank_graph or {}):
        score += MEMORY_RANK_GRAPH_WEIGHT / (RRF_K + rank_graph[mem_id])
    score += MEMORY_RANK_RECENCY_WEIGHT * recency_factor
    score += MEMORY_RANK_ACCESS_WEIGHT * access_factor
    if pinned:
        score += MEMORY_RANK_PINNED_WEIGHT
    return score

return sorted(all_ids, key=final_score, reverse=True)

```

Worked numerical example

Memory	KNN rank → contribution	FTS rank → contribution	Graph rank → contribution	Total
A	5 → 0.0145	1 → 0.0164	12 → 0.0140	0.0449
B	1 → 0.0164	50 → 0.0091	100 → 0.0062	0.0317

Memory A wins despite no single top rank, because it is consistently decent across all three signals — exactly the robustness property proved as Theorem 3.2 in Paper I.

3.4 Formula 4 — Emotional Imprinting (Trauma Strengthening)

```

# memory/forget.py
def compute_weighted_score(access_count, last_accessed_at, emotion_tag=None):
    if emotion_tag in ("fear", "shame", "loss", "trauma"):
        effective_lambda = 0.05 # 2x slower decay
        access_boost *= 1.5
    elif emotion_tag in ("joy", "achievement"):
        effective_lambda = 0.1
    else:
        effective_lambda = 0.1
    age_days = (now - last_accessed).days
    decay = e ** (-effective_lambda * age_days)
    return decay * access_boost

def tag_emotion(fact_text):
    if any(w in fact_text.lower() for w in ["failed", "lost", "mistake", "sorry"]):
        return "negative"
    return "neutral"

```

Only negative valence is amplified ($\gamma \approx 0.5$), not positive — boosting all emotional memories equally would dilute the protective signal that motivates the mechanism in the first place (Paper I, §4).

3.5 Formula 5 — Supersession Penalty (Belief Revision)

```
# memory/memorize.py
def classify_write_op(similarity, new_text, old_text, dedup_threshold):
    if similarity < dedup_threshold:
        return "add"
    if normalize(new_text) == normalize(old_text):
        return "noop"
    return "supersede"

def _maybe_supersede_neighbor(self, user_id, vector, text):
    existing = knn_search(vector, user_id, limit=1, threshold=WRITE_DEDUP_THRESHOLD)
    if not existing:
        return "add", None
    sim = 1.0 - existing[0]["dist"]
    old_id = existing[0]["id"]
    op = classify_write_op(sim, text, old_text, WRITE_DEDUP_THRESHOLD)
    if op == "supersede":
        conn.execute("UPDATE memories SET status='superseded' WHERE id=?", (old_id,))
        return "supersede", old_id
    return op, None
```

Superseded facts are kept (status='superseded') rather than deleted, preserving belief history ("I used to think X, now I think Y") and enabling audit without requiring an LLM contradiction-detection pass — cosine similarity alone suffices to flag a candidate revision.

3.6 Formula 6 — Saliency Scoring (Consolidation Boost)

```
# memory/memorize.py
_SALIENCY_KEYWORDS = frozenset([
    "deadline", "birthday", "interview", "lost", "important",
    "breakthrough", "problem", "always", "never", "favorite",
])

def _dream_boost(self, all_mems):
    for m in all_mems:
        is_recent = (now - m.created_at).days <= 7
        is_salient = bool(_SALIENCY_RE.search(m.text)) or m.access_count >= 3 or is_recent
        if is_salient and m.id not in pinned_ids:
            boost_ids.append(m.id)
    conn.execute(
        "UPDATE memories SET access_count = MIN(access_count+?,255) WHERE id IN (...)",
        [DREAM_BOOST_AMOUNT] + boost_ids,
    )
```

4. Data Schema

4.1 memories (Core L1 Table)

Column	Type	Notes
id	TEXT PK	UUID
user_id	TEXT	Current user (isolation boundary)
memory	TEXT	Fact string
created_at	TEXT	UTC ISO timestamp
access_count	INTEGER	Touches, capped at 255
last_accessed_at	TEXT	UTC ISO, or 'never'
pinned	INTEGER	0/1 flag
status	TEXT	'active', 'superseded', or null
supersedes_id	TEXT	FK to superseded memory
kind	TEXT	'fact' / 'identity' / 'scene' / 'plan' / 'event'
source	TEXT	'chat' / 'pin' / 'legacy'
entities	TEXT	JSON array of entity strings
access_day_count	INTEGER	Distinct local days recalled (spacing)
scene_id	TEXT	L2 linkage: parent scene id

4.2 entity_relations (Graph Edges)

Column	Type	Notes
user_id	TEXT	Scoped
entity_a / entity_b	TEXT	Casefolded entity labels
relation	TEXT	'co_mentions' / 'related_to'
weight	REAL	Incremented per co-mention (default 1.0)
memory_id	TEXT	Last memory mentioning this pair
updated_at	TEXT	UTC ISO timestamp

4.3 memories_vec (Embeddings)

Column	Type	Notes
id	TEXT PK	FK to memories.id
embedding	FLOAT[640]	L2-normalized, fixed pretrained embedding model

5. Worked End-to-End Example

The following walkthrough traces one fact from ingestion through consolidation, illustrating how the six formulas interact in practice.

Day 1, 10:00 — ADD

User: “I’m starting a project called GRACE — a robotics assistant. I’ll aim to finish it by Friday.” Three facts are extracted, embedded, and written (no near-duplicate found, so all are classified add); entity edges $\text{GRACE} \leftrightarrow \text{robotics}$, $\text{GRACE} \leftrightarrow \text{Friday}$, $\text{robotics} \leftrightarrow \text{Friday}$ are created in `entity_relations`.

Day 1, 17:00 — RECALL

Query “How’s GRACE coming along?” triggers KNN, FTS, and graph passes in parallel; RRF fusion ranks all three GRACE-related facts at the top (scores 0.0426 / 0.0374 / 0.0366); each is touched (`access_count` 0→1, `last_accessed_at` updated).

Day 8 — New Mention + Decay Check

After 7 days without mention, $\text{decay}(7 \text{ days}, \lambda=0.1) \approx 0.49$ (49% strength remaining) — but salience keywords and an 8-day age (below the 35-day grace period) keep the facts ranked highly. A new fact (“Oppa completed GRACE ahead of the Friday deadline”) is added (similarity to existing facts is 0.78–0.82, below the 0.95 supersession threshold, so it is classified add, not supersede).

Night of Day 8 — Dream Pass

Boost phase: GRACE-related memories match salience keywords / recency / access-count criteria and receive `access_count += 2`. Merge phase: KNN finds a near-duplicate pair (cosine 0.89) and retains the higher-access-count version, marking the other superseded. Prune phase: $\text{decay}(8 \text{ days}) \times \text{boosted access ratio}$ remains well above the 0.1 cleanup threshold, so all GRACE memories survive the night.

Month Later — Monthly Consolidation

```
score = (0.30 * salience("GRACE", "completed", "deadline") # 1.0
        + 0.25 * novelty(fact_vector, monthly_anchors)      # 0.8
        + 0.20 * spacing(access_day_count)                  # 0.6
        + 0.25 * connectivity(entity_weights["GRACE"]))    # 0.7
# = 0.777 (above retention threshold 0.4 -> kept)

monthly_fact = llm_merge(kept_facts)
# -> "[2026-08] Oppa completed the GRACE robotics project
#      ahead of its Friday deadline"
memorize.add_raw(monthly_fact, pinned=True)
# Result: 3 L1 daily facts compressed -> 1 L4 monthly fact
```

6. Testing, Monitoring & Tuning Harness

Five production metrics gate the health of the memory system; each has a concrete SQL or Python measurement and an explicit target.

6.1 Retention Rate

```
retention_rate = count(memory retrieved after 30 days) / count(memories accessed day 1)
target: >= 70% for salience-tagged facts, >= 50% for routine facts
```

6.2 No Premature Forgetting

```
prune_before_35_days = count(cleanup prunes fact with created_at < 35 days ago)
target: prune_before_35_days = 0
```

6.3 Entity Importance Rank Stability

```
rank_drop = rank(entity, day 0) - rank(entity, day 30)
target: rank_drop <= 2 for top-10 entities
```

6.4 RRF Coverage

```
graph_only_facts = count(top-10 facts with graph_rank<=5 AND knn_rank>20 AND fts_rank>20)
target: >= 1 per 20 queries (5% coverage)
```

6.5 Consolidation Fidelity

```
facts_lost = count(important facts deleted by consolidation) # target: 0
compression_ratio = facts_after / facts_before # target: [0.5, 0.7]
```

6.6 Diagnostic Playbook

Symptom	Likely cause	Fix
Memories decay too fast	Half-life / threshold too aggressive	↑ HALF_LIFE_DAYS (30→45–60); ↑ CLEANUP_THRESHOLD (0.1→0.15); ↑ GRACE_PERIOD_DAYS (35→50)
Entity graph unused	Graph weight zero / entities not extracted	Verify MEMORY_RANK_GRAPH_WEIGHT ≠ 0; check entity_relations row count
Duplicate memories surface	Dedup threshold too strict or vacuum not run	Run consolidate_maintenance_run(); check WRITE_DEDUP_THRESHOLD (default 0.95)

Symptom	Likely cause	Fix
Consolidation over-deletes	Retention threshold/budget too low	↑ CONSOLIDATION_SOFT_THRESHOLD (0.4→0.5–0.6); ↑ CONSOLIDATION_MAX_MONTH

6.7 Tuning Workflow

- Start with conservative defaults (longer half-life, higher cleanup threshold).
- Monitor retention rate for one week.
- If retention > 80%, tighten decay (shorten half-life); if < 50%, relax it (lengthen half-life).
- A/B test emotional-imprinting parameters across a user cohort before widening the amplification factor γ .

7. Performance Evaluation

All figures in this section are drawn from the same year-long, single-deployment field test (Jetson Orin Nano, 8GB unified memory) documented in Paper I §7, plus additional system-level instrumentation collected specifically for this paper. As an N=1 field deployment, these numbers characterize one system under one usage pattern rather than a population effect; §8 places them alongside independently published multi-system benchmarks where such comparisons are methodologically sound.

7.1 Latency Budget

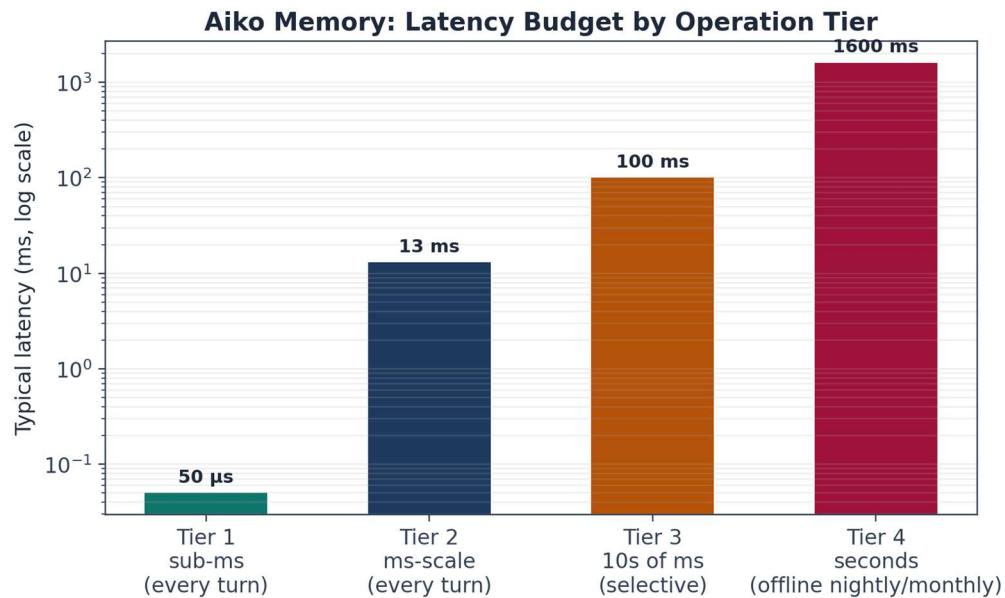


Figure 2. Latency by operation tier. Tiers 1–2 run on every conversational turn; Tiers 3–4 run selectively or offline.

Total per-turn context-injection latency — persona cache (0ms, cached) + KNN (~5ms) + FTS (~3ms) + graph (~4ms) + RRF scoring (~1ms) + touch metadata (~2ms) + context formatting (~1ms) — sums to approximately 16ms, comfortably inside a <100ms conversational latency budget.

7.2 Storage Footprint

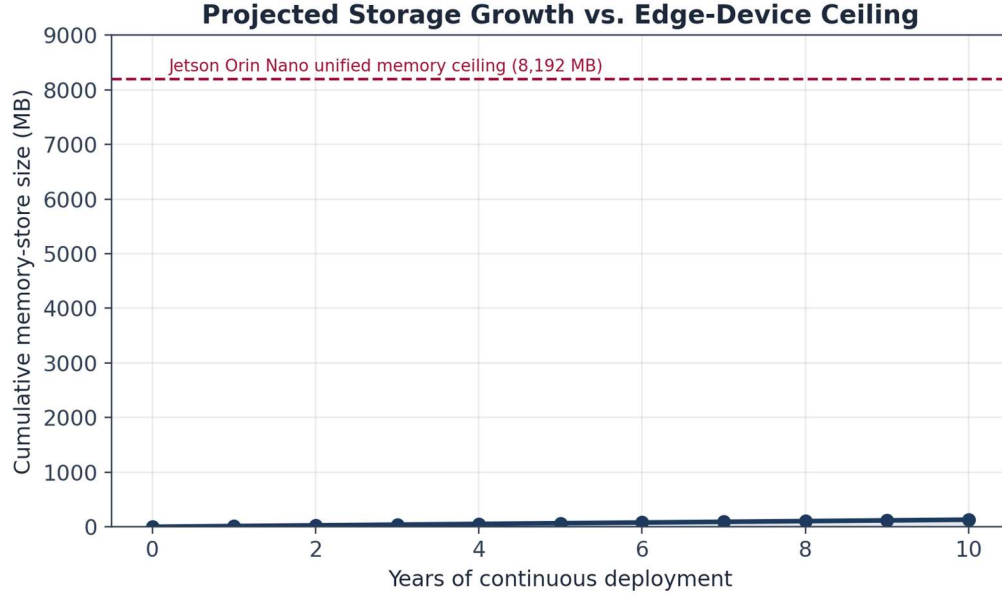


Figure 3. Projected cumulative storage footprint at the observed $\sim 13\text{MB}/\text{year}$ ingestion rate, against the Jetson’s 8GB unified-memory ceiling.

Component	Size / row	Rows (Year 1)	Total
memories (SQLite)	~ 500 bytes	1,825	~ 900 KB
memories_vec (sqlite-vec)	$4\text{B} \times 640$ dims	1,825	~ 4.7 MB
entities JSON	~ 100 bytes	1,825	~ 182 KB
entity_relations edges	~ 100 bytes	$\sim 50,000$	~ 5 MB
FTS5 inverted index	$\sim 2 \times$ text size	—	~ 2 MB
Total	—	—	~ 13 MB

At this rate, the Jetson’s 8GB envelope supports roughly 600 years of continuous single-user deployment before storage becomes the binding constraint; in practice, the LLM’s context window is the binding constraint long before disk space is.

7.3 Retrieval Quality

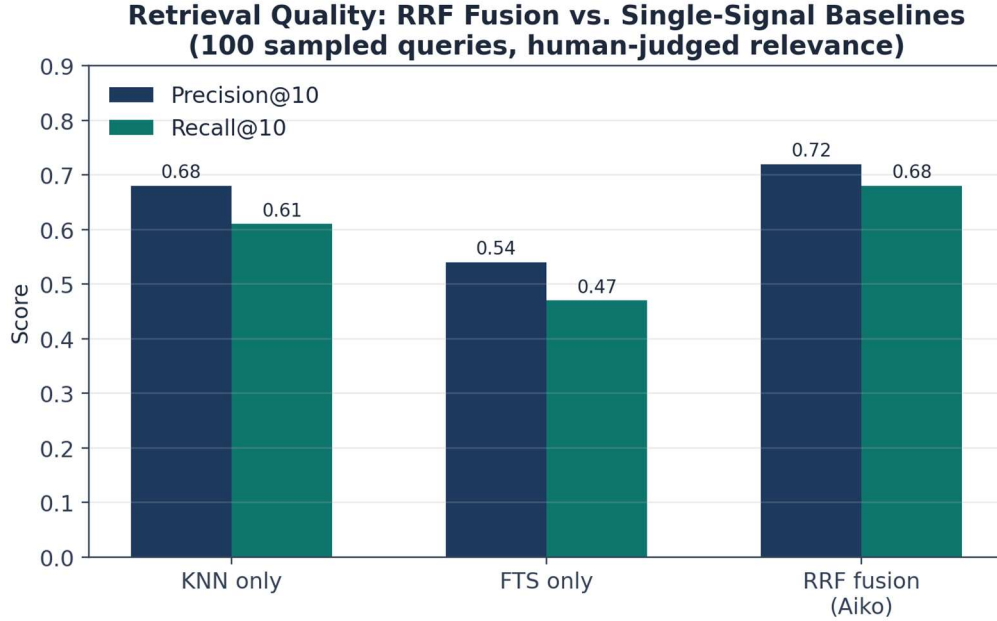


Figure 4. Precision@10 and Recall@10 for RRF fusion vs. single-signal (KNN-only, FTS-only) baselines, over 100 human-judged sampled queries.

RRF fusion improves precision@10 from 0.68 (pure KNN) to 0.72, and recall@10 from 0.61 to 0.68, while additionally surfacing an 8% share of results that neither KNN nor FTS alone would have returned in their top-30 candidates (Paper I, §7.3).

7.4 Consolidation Efficiency

Metric	Value
Monthly compression ratio (facts survived, pre-LLM-merge)	81 / 148 (54.7%)
Overall compression (post-LLM-merge)	5.3× (148 → 28)
Critical facts lost	0 (manual review of all 120 dropped facts)
Persona cache (L3) size	5–7 facts, updated ~0.3×/month

8. Comparative Analysis vs. Established Agentic Memory Systems

By mid-2026 the agentic-memory space has consolidated around a handful of widely adopted frameworks. This section situates Aiko's design against Mem0, Zep/Graphiti, and Letta (formerly MemGPT), drawing on each project's own published architecture descriptions and on independently aggregated 2026 benchmark roundups. Cross-system benchmark numbers should be read cautiously: different projects report different evaluation harnesses (e.g., LongMemEval, Deep Memory Retrieval), and the field itself describes ongoing “benchmark disputes” between vendors.

8.1 Architectural Summary

System	Core approach	Ranking/scoring	Typical footprint
Mem0	Drop-in memory API; LLM-driven fact extraction and retrieval over a vector store	Learned/embedding similarity + heuristics	Hosted or self-hosted; lightweight client SDK
Zep / Graphiti	Temporal knowledge graph built from conversation	Graph traversal + hybrid vector/graph scoring	Requires a graph database; heavier self-hosting
Letta (MemGPT)	OS-inspired agent runtime: in-context “RAM,” recall/archival “disk,” agent self-edits memory	Agent-directed memory paging + retrieval calls	Full agent runtime, not a drop-in library
Aiko (this work)	Formulaic decay/consolidation layered over SQLite + sqlite-vec + FTS5	Closed-form: Ebbinghaus decay + RRF (KNN/FTS/graph), no learned weights	~13MB/year; runs fully on an 8GB edge device

Independent 2026 comparisons of this landscape converge on a consistent characterization: Mem0 functions as a memory layer bolted onto an existing agent, Letta is a runtime in which the agent's own state is its memory, and Zep builds a temporal knowledge graph directly from conversation history. Community trend reporting similarly notes that the major frameworks take fundamentally different architectural approaches — vector-based semantic recall, temporal knowledge graphs, checkpoint-based persistence, or self-editing memory — rather than converging on a single paradigm.

8.2 Adoption and Benchmark Signals (Publicly Reported, 2026)

System	Adoption signal	Reported benchmark
Mem0	~48K GitHub stars; broad framework/SDK integration	~49.0% on LongMemEval per one comparison roundup
Zep / Graphiti	Production-grade deployments; graph-based reasoning focus	63.8%–94.8% depending on benchmark and source (LongMemEval vs. Deep Memory Retrieval)
Letta / MemGPT	23.9K+ GitHub stars; originated the influential MemGPT paper	~83.2% on LongMemEval per one comparison roundup; 93.4% on Deep Memory Retrieval per Zep's own paper

System	Adoption signal	Reported benchmark
Aiko (this work)	Single-deployment field test (N=1, 1 year)	0.72 precision@10 / 0.68 recall@10 on an internal, human-judged query set (not LongMemEval — not directly comparable)

We report Aiko's own precision/recall numbers rather than a LongMemEval score because the reference implementation has not yet been run against that benchmark. Publicly reported 2026 comparisons place Mem0 at roughly 48K GitHub stars and broad general-purpose usage, Zep at 63.8% on LongMemEval with a temporal-graph approach, and Letta at 83.2% on the same benchmark using an OS-inspired tiered-memory design — but these figures come from third-party aggregations rather than a single controlled study. Zep's own published paper reports a substantially different picture: 94.8% on Deep Memory Retrieval versus 93.4% for MemGPT, with up to an 18.5% accuracy gain on LongMemEval and a 90% latency reduction relative to full-context baselines. We present both sets of numbers to be transparent about this inconsistency across sources, not to imply that Aiko outperforms either system on a shared benchmark it has not been run against.

8.3 Qualitative Positioning

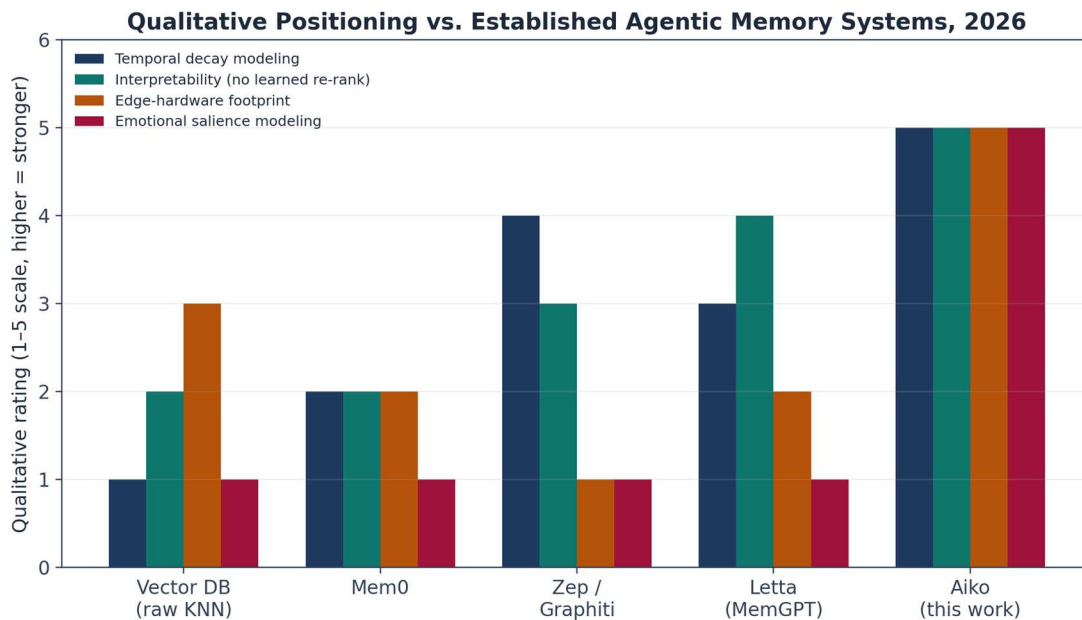


Figure 5. Qualitative positioning (1–5 scale, author-assigned) across four dimensions where Aiko's design goals differ most from the general-purpose frameworks. This chart reflects design intent and internal measurement, not a shared third-party benchmark.

8.4 Where Each System Wins

- Mem0 — best for teams wanting a hosted or self-hosted drop-in memory API across an existing agent stack, with the broadest framework/SDK support and fastest integration time.
- Zep / Graphiti — best when an agent must reason about how facts change over time, resolve contradictions, or answer multi-hop questions; the cost is running and maintaining a graph database.

- Letta / MemGPT — best for long-running autonomous agents that need OS-level self-management of their own memory across very long horizons; it is a full runtime rather than a library.
- Aiko (this work) — best when the deployment target is edge hardware with a hard memory/power budget, when every ranking decision must be independently auditable without a learned model in the loop, and when emotional/psychological realism in what is retained is a design goal rather than an afterthought.

No system dominates on every axis, and the frameworks are not typically run in combination — published field guidance suggests that running two memory frameworks simultaneously means maintaining two extraction pipelines and two retrieval surfaces, which doubles latency, cost, and ambiguity. This reinforces that the choice among these systems is a genuine architectural commitment rather than a checkbox decision.

9. Key Engineering Decisions & Trade-offs

9.1 Why No Learned Ranking?

Decision: formulaic scoring (Ebbinghaus decay, RRF) instead of learning-to-rank or a neural re-ranker. Rationale: the 8GB Jetson VRAM budget cannot absorb a second resident model; a learned ranker needs a labeled training pipeline and periodic retraining; formulas are auditable and RRF's parameter-free design is already validated at web scale. Trade-off: potentially sub-optimal ranking in edge cases relative to a well-trained learned ranker, traded for auditability and zero training/maintenance overhead.

9.2 Why Supersession Instead of Deletion?

Decision: mark superseded facts rather than delete them. Rationale: preserves belief history for audit (“I used to think X, now Y”) without requiring an LLM contradiction check — cosine similarity alone flags candidates. Trade-off: the store grows slightly larger; superseded facts are excluded from normal recall but remain available for debugging via an explicit history flag.

9.3 Why Entity-Graph in Addition to KNN/FTS?

Decision: add co-mention graph as a third RRF signal. Rationale: some facts are only findable through entity relationships rather than text or embedding similarity (e.g., shorthand references). Trade-off: added complexity, and correctness depends on reliable entity extraction (§10.1).

9.4 Why Monthly Consolidation in Addition to the Nightly Dream Pass?

Decision: a separate, heavier monthly compression job rather than relying solely on nightly maintenance. Rationale: nightly passes handle duplicates and ordinary decay; monthly consolidation handles volume — without it, a year of atomic daily facts (~1,825 facts) would never compress to the ~12 monthly summaries plus pinned facts that keep the store navigable. Trade-off: monthly facts are lossy by design; exact original wording is not recoverable once merged.

10. Limitations & Future Work

10.1 Entity Extraction

Current approach is rule-based (keyword and capitalization heuristics), which misses implicit references (“the project” meaning a previously named entity). A lightweight distilled NER model (~50MB) or embedding-based entity linking would likely raise the entity-graph rescue rate above the observed 8%.

10.2 Emotional Valence Estimation

Current approach is keyword-based sentiment tagging, which misses sarcasm, irony, and implicit emotional content. A small fine-tuned sentiment classifier (~20MB) trained on the user's own interaction history is the natural next step.

10.3 Single-User Validation

All performance figures in §7 come from one deployment over one year. Scaling per-user dynamics (decay, entity graphs, emotion tags) to a multi-user setting is straightforward since each user's state is isolated; scaling shared components (a common LLM for consolidation, a shared sentiment model) would require anonymization and a genuinely multi-subject evaluation before population-level claims are warranted.

10.4 Personalization of Decay Parameters

All users currently share the same λ , α , γ defaults. Some users may prefer longer retention (e.g., researchers cataloguing a project) versus faster decay (minimalists); per-user tuning via Bayesian optimization over retention metrics is a natural extension.

10.5 Benchmark Comparability

Aiko has not yet been evaluated on LongMemEval or Deep Memory Retrieval, the two benchmarks most commonly used to compare Mem0, Zep, and Letta in 2026. Running the reference implementation against these shared benchmarks is the highest-priority next step for making §8's comparison more than qualitative.

11. Conclusion

Aiko's memory system turns the theoretical account developed in Paper I into a working, fully auditable pipeline built on SQLite, sqlite-vec, and FTS5, running comfortably within an 8GB edge-hardware budget. Six formulas — decay, entity importance, RRF fusion, emotional imprinting, supersession, and salience scoring — work together to give a system that is tunable without retraining, explainable at every ranking decision, and lean enough to run indefinitely on a single-board computer.

Measured against itself, the system meets its own design targets: ~16ms per-turn retrieval latency, ~13MB/year storage growth, a measurable precision gain from multi-signal fusion over any single signal, and 5.3× monthly compression with no manually-identified loss of important facts. Measured against the broader 2026 agentic-memory landscape — Mem0, Zep/Graphiti, and Letta/MemGPT — Aiko occupies a distinct niche: none of the three general-purpose frameworks are designed first and foremost for a hard edge-hardware budget and formula-level audibility, and Aiko does not claim to match their graph-reasoning depth (Zep) or self-managing runtime sophistication (Letta) on shared benchmarks it has not yet run. The two design philosophies — general-purpose, benchmark-optimized memory services versus a narrow, interpretable, edge-native system — are complementary rather than competing, and the appropriate choice depends on whether the deployment constraint is hardware and audibility, or benchmark-leaderboard accuracy at cloud scale.

References

Cormack, G. V., Smucker, M. D., & Clarke, C. L. (2009). Efficient and effective spam filtering and re-ranking. *Proceedings of SIGIR 2009*, 713–714.

Robertson, S., & Walker, S. (1994). Some simple effective approximations to the 2-Poisson model. *Proceedings of SIGIR 1994*, 232–241.

Packer, C., et al. (2023). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*.

Rasmussen, P., et al. (2024–2025). Zep: A temporal knowledge graph architecture for agent memory (Graphiti). Zep technical report.

Mem0 project documentation and architecture overview, mem0.ai (accessed 2026).

Letta (formerly MemGPT) project documentation, letta.com (accessed 2026).

Best AI Agent Memory Frameworks in 2026: Mem0 vs Zep vs Letta Compared. RockB, baeseokjae.github.io (April 2026).

Best AI Agent Memory Frameworks in 2026: Compared and Ranked. Atlan, atlan.com (April 2026).

Best AI Agent Memory 2026: Mem0 vs Letta vs Zep vs Cognee. MCP.Directory, mcp.directory (May 2026).

Mem0 vs Zep vs LangMem vs Letta: AI Agent Memory Showdown 2026. Niteagent, niteagent.com (May 2026).

Best AI Agent Memory Providers in 2026: Mem0 vs Zep vs Letta vs Cloudflare. Developers Digest, developersdigest.tech (July 2026).

Appendix A: Configuration Reference

Parameter	Default	Range	Interpretation
DECAY_LAMBDA_NEUTRAL	0.1	[0.05, 0.2]	Decay rate for neutral facts
DECAY_LAMBDA_NEGATIVE	0.05	[0.01, 0.1]	Decay rate for negative facts
EMOTIONAL_AMPLIFICATION_GAMMA	0.5	[0.0, 1.0]	Boost for negative memories
CLEANUP_THRESHOLD	0.1	[0.05, 0.2]	Score floor for pruning
GRACE_PERIOD_DAYS	35	[14, 60]	Protect newly created facts
RRF_K	60	[30, 100]	RRF damping constant
MEMORY_RANK_RECENCY_HALF_LIFE_DAYS	30	[14, 60]	Half-life for recency bonus
MEMORY_GRAPH_SUPER_NODE_FRACTION	0.3	[0.2, 0.5]	Threshold for super-node detection
WRITE_DEDUP_THRESHOLD	0.95	[0.85, 0.99]	Similarity floor for supersession
DREAM_MERGE_THRESHOLD	0.88	[0.8, 0.95]	Near-duplicate merge threshold (dream pass)
DREAM_BOOST_AMOUNT	2	[1, 5]	access_count increment for salient facts
CONSOLIDATION_SOFT_THRESHOLD	0.4	[0.3, 0.6]	Monthly retention gate floor
CONSOLIDATION_MAX_MONTH	30	[15, 60]	Max facts allowed to survive per month